

Programação Concorrente

Trabalho 1

Willyan Marques de Melo, 221020940

¹Dep. Ciência da Computação – Universidade de Brasília (UnB)
CIC0202 - Programação Concorrente
25 de junho de 2026

willyanmarquesmelo@gmail.com

1. Introdução

O presente relatório irá propor e analisar um algoritmo de sincronização elaborado para resolver um problema de comunicação entre processos e threads que compartilham memória através de uma estrutura de árvore binária. O principal desafio é fazer com que a sincronização obedeça à hierarquia da árvore e ocorra de baixo para cima (bottom-up).

A formalização do problema teve como inspiração o algoritmo de merge sort, que é capaz de diminuir a complexidade computacional da organização de vetores por meio da recursão, divisão e conquista e por paralelismo. Além disso, usou-se também de inspiração o conceito de “harmonia musical”, que é capaz de fazer uma analogia direta com a sincronização de threads. Ao considerarmos um coro ou um conjunto instrumental, para que tenhamos harmonia, é necessário que haja a sincronia deles dentro de um compasso e ritmo específicos.

É traçado, então, um paralelo entre teoria musical e concorrência de processos: se considerarmos que as vozes são threads, podemos reduzir a abstração para algo audível no mundo real. Portanto, o problema propõe que seja feita a harmonização entre diversas faixas de áudio executadas simultaneamente e que possuem velocidades de reprodução inicial aleatórias, por meio dos diversos mecanismos de sincronização oferecidos pela biblioteca POSIX thread da linguagem C.

2. Formalização do Problema Proposto

Considere um coro musical formado por um grupo de $2^{k+1} - 1$ pessoas organizadas hierarquicamente na forma de uma árvore binária perfeita de altura k , em que cada integrante desse coro é representado por um nó desta árvore, o qual controla uma faixa de áudio. Infelizmente, por não terem praticado juntos anteriormente, todas as pessoas começam a cantar em uma velocidade aleatória que varia de 80 a 120 BPM (batidas por minuto).

O seu objetivo é desenvolver um programa concorrente na linguagem C, utilizando a biblioteca POSIX Pthreads, para sincronizar o BPM de todos os cantores, propagando os ajustes de BPM da árvore de baixo para cima (bottom-up). O programa deve ter como entrada um número inteiro k representando a altura da árvore binária, determinando o número total de integrantes do coro. Utilize logs no console para demonstrar o mecanismo de sincronização e comunicação feita pelas threads.

O comportamento esperado é que todas as faixas de áudio começam a tocar de maneira caótica e as faixas de áudio são sincronizadas uma subárvore por vez, até que

todos estejam com a mesma velocidade no mesmo instante de reprodução. A solução deve obedecer às seguintes restrições:

- Cada faixa de áudio inicia sua reprodução com um valor de batidas por minuto (BPM) definido aleatoriamente (entre 80 e 120 BPM), operando de maneira assíncrona.
- A sincronização e comunicação entre fluxos de execução para harmonização da música devem ocorrer apenas entre o nó pai e seus filhos, partindo das folhas até chegar à raiz.
- Não é permitida a utilização de um mutex ou barreira global para sincronizar todos os nós de uma só vez.
- O mecanismo de sincronização não pode interromper a reprodução das faixas de áudio.
- O algoritmo deve ser livre de deadlocks e condições de corrida.

3. Descrição do Algoritmo Desenvolvido

O algoritmo utiliza locks, variáveis de condição e barreiras disponibilizados na biblioteca Pthreads. Ele usa como base duas estruturas em C (*AudioTrack* e *TreeNode*) para guardar informações do nó (dados internos do nó, identificador da thread e mecanismos de sincronização) e o estado da faixa de áudio (BPM e estado local da reprodução). Cada *TreeNode* dessa árvore possui as seguintes propriedades:

```
1 typedef struct TreeNode {
2     int id; // Identificador da thread
3     int is_root; // 1 é raiz, 0 não é raiz
4     int is_leaf; // 1 é folha, 0 não é folha
5     int level; // Qual o nível/altura do nó
6     struct TreeNode* parent; // Ponteiro para o pai (travessia bottom-up)
7     struct TreeNode* left; // Ponteiro para o filho à esquerda
8     struct TreeNode* right; // Ponteiro para o filho à direita
9     pthread_barrier_t barrier; // Barreira para sincronização
10    AudioTrack track; // Informações para thread de áudio
11 } TreeNode;
```

Figura 1. Definição struct *TreeNode*

O *id* é usado para identificação da thread de sincronização e indexação do nó, enquanto *is_root* e *is_leaf* são usados para informar se o nó é raiz ou folha, respectivamente. O *level* indica a qual nível da árvore o nó pertence. Os ponteiros *parent*, *left*, *right* indicam o nó pai, o nó à esquerda e o nó à direita; o ponteiro para o pai foi necessário para a travessia bottom-up ser possível. Cada nó tem uma barreira que será utilizada para sincronização, a qual será explicada posteriormente. Por fim, cada nó possui uma struct *AudioTrack* com os dados necessários para reprodução e controle das faixas de áudio:

```

1 typedef struct AudioTrack {
2     int id; // Identificador da thread
3     int bpm; // Velocidade de reprodução
4     int updated; // Flag para sincronização (espera continuada)
5     double accumulated_time; // Duração que a faixa tocou
6     struct timespec last_play_time; // Tempo em que a faixa começou a última reprodução
7     pthread_mutex_t lock; // Lock para atualização dos dados
8 } AudioTrack;

```

Figura 2. Definição struct AudioTrack

Novamente, o id é usado para identificação da thread de áudio e indexação na árvore, o BPM é a velocidade atual de reprodução, updated é uma *flag* para realização de espera continuada (explicação à frente), *accumulated_time* e *last_play_time* são usados para continuar a reprodução sem reiniciar a faixa e o lock é utilizado para atualizar todos esses dados e evitar condições de corrida.

Visto que uma das condições do problema é que a reprodução do áudio não seja interrompida, é necessário um desacoplamento entre o mecanismo de sincronização e o mecanismo de reprodução, logo para cada nó teremos uma thread de sincronização (sync_thread) e uma thread de áudio (audio_thread).

3.1. Reprodução de Áudio (audio_thread)

As threads responsáveis pela reprodução do áudio têm um comportamento bastante simples: elas iniciam a reprodução por meio da função `play_track` e aguardam até que o seu BPM seja alterado. Como o foco era harmonizar os áudios, optou-se por utilizar um mecanismo de espera contínua para uma única pthread_cond (`atualizar_tracks_cond`), de modo que todas as threads acordam a partir de um único broadcast. A flag `update` é utilizada como controle para saber se a thread de áudio deve ou não alterar o seu BPM.

Observe também que existe uma barreira global **antes do início da sincronização da árvore**. Essa barreira foi utilizada também em `sync_thread` para garantir que todas faixas de áudio estejam tocando antes da sincronização começar, logo essa barreira não quebra as restrições estabelecidas.

A função `play_track` utiliza a biblioteca `ffplay` para executar comandos via `system()` e reproduzir arquivos “.mp3”. Ao ser executada, ela mata a thread de áudio anterior no sistema e reinicia a reprodução a partir do último tempo registrado pelo nó pai no update dos BPMs.

```

1 void* audio_thread(void* arg) {
2     int id = *((int*)arg);
3     TreeNode* node = tree[id];
4     AudioTrack* track = &(node->track);
5
6     printf("[Audio %d] Iniciado. BPM Inicial: %d. Indo dormir... \n", id, track->bpm);
7
8     play_track(track); // Play
9     pthread_barrier_wait(&global_start_barrier); // Barreira inicial para garantir que a reprodução começa antes da sincronização
10
11 // A única coisa que essa thread faz é reproduzir o áudio e reiniciar a reprodução caso o BPM mude
12 while (run) {
13     pthread_mutex_lock(&track->lock); // Lock para espera cotinuada
14     {
15         printf("[Audio %d] Aguardando broadcast ... \n", id);
16         while (!track->updated && run) { // Loop necessário para simplificar broadcast
17             pthread_cond_wait(&atualizar_tracks_cond, &track->lock); // Fica em espera até que ocorra uma nova atualização do BPM
18         }
19         printf("[Audio %d] Novo BPM detectado: %d\n", id, track->bpm);
20         track->updated = 0; // Flag para espera
21     }
22     pthread_mutex_unlock(&track->lock);
23
24     play_track(track); // Replay
25 }
26
27 return NULL;
28 }

```

Figura 3. Definição audio_thread

3.2. Sincronização Hierárquica (sync_thread)

Como dito anteriormente, cada thread tem um nó de referência na árvore e cada nó tem uma barreira. Essas barreiras têm um contador de 3 threads, com exceção das folhas cujo contador é para apenas 1 thread (sincronização inicial local). Portanto, primeiro devemos analisar o caso de execução base em que $k=0$, ou seja, a árvore só possui um nó que é raiz e folha ao mesmo tempo, o que significa que não tem o que sincronizar. Se a árvore possui mais de um nó, então existirão dois tipos: nós com filhos e nós sem filhos (nós folha). Caso o nó seja uma folha, então as threads não têm com quem sincronizar, logo elas começam localmente sincronizadas e só precisam acessar a próxima barreira na hierarquia.

```

1 void* sync_thread(void* arg) {
2     int id = *((int*)arg);
3     TreeNode* node = tree[id];
4     AudioTrack* track = &(node->track);
5
6     pthread_barrier_wait(&global_start_barrier);
7
8     printf("[Sync %d] Iniciado. %s\n", id, node->is_leaf ? "É folha." : (node->is_root ? "É raiz." : "Nó intermediário."));
9
10 // 1. CASO BASE: um único nó que é folha e raiz ao mesmo tempo
11 if (node->is_leaf && node->is_root) {
12     printf("[Sync %d] É folha e raiz ao mesmo tempo (N=0). Não há com quem sincronizar. Encerrando.\n", id);
13     return NULL;
14 }
15
16 // 2. SE FOR APENAS FOLHA
17 if (node->is_leaf) {
18     // Começa "sincronizado", acesso a barreira do pai imediatamente e que começa a propagação bottom-up
19     printf("[Sync %d] Esperando na barreira do pai (nó %d)... \n", id, node->parent->id);
20     pthread_barrier_wait(&node->parent->barrier);
21     return NULL;
22 }

```

Figura 4. Sincronização caso base e caso folha.

Agora, se o nó não for folha, ele ainda pode ser raiz ou intermediário, de qualquer

forma, eles seguem a mesma lógica. As threads ligadas a esses nós executam a lógica de sincronização e quatro fases:

1. **Espera:** a thread espera na própria barreira pelas threads dos nós filhos (caso existam). Isso faz com que a sincronização ocorra obrigatoriamente de baixo para cima. Quando os filhos liberam a barreira local, a thread avança para a próxima fase.
2. **Cálculo e propagação:** a thread executa a função `sincronizar_subarvore` que por sua vez calcula um novo BPM com base nas subárvores do nó informado - como o algoritmo é bottom-up, é garantido que as sub-árvores à esquerda e a direita já estão sincronizadas - e em seguida chama a função `atualizar_bpm_recurso` que propaga esse novo valor para todos os AudioTracks da subárvore deste nó.
3. **Atualização e replay:** após atualizar os BPMs, é feito um broadcast para as threads de áudio para elas reiniciem na reprodução com essa nova velocidade, usando como referência o tempo de reprodução do áudio da raiz da subárvore local.
4. **Verificação e Liberação:** por fim, a thread verifica se seu nó é raiz: em caso positivo ele encerra o programa pois todos nós foram sincronizados; senão ele acessa a barreira do nó pai para liberar a próxima thread da hierarquia.

```
1 // 3. SE FOR INTERMEDIÁRIO OU RAIZ
2
3 // Primeira fase (espera para sincronização)
4 printf("[Sync %d] Esperando na PRÓPRIA barreira os filhos chegarem...\n", id);
5 pthread_barrier_wait(&node->barrier); // Espero na própria barreira
6 printf("[Sync %d] Barreira liberada! Vou calcular a média da minha subárvore.\n", id);
7
8 // Segunda fase (cálculo do BPM e atualização da subárvore)
9 sincronizar_subarvore(node);
10
11 // Terceira fase (broadcast)
12 printf("[Sync %d] ENVIANDO SINAL DE UPDATE\n", id);
13 pthread_cond_broadcast(&atualizar_tracks_cond); // Faz broadcast para as thread de audio reiniciarem a reprodução
14
15 // Última fase (verificação)
16 if (!node->is_root) { // Se for nó intermediário, acesso a próxima barreira
17     printf("[Sync %d] Intermediário terminou. Esperando na barreira do pai (nó %d)...\n", id, node->parent->id);
18     pthread_barrier_wait(&node->parent->barrier);
19 } else { // Se for a raiz, acabou a sincronização
20     printf("[Sync %d] É RAIZ. Sincronização completa alcançada.\n", id);
21 }
22
23 return NULL;
24 }
```

Figura 5. Caso que o nó tem filhos.

A atualização do BPM feita na segunda fase foi encapsulada na função `sincronizar_subarvore` para melhor legibilidade, mas sua definição segue abaixo. A função `atualizar_bpm_recurso` apenas percorre a árvore a partir do nó de referência atualizando os valores de AudioTrack de cada nó.

```

1 void sincronizar_subarvore(TreeNode* node) {
2     sleep(((N - node->level) * 2) + (rand() % 5)); // Delay para conseguir ouvir sincronizando (inversamente proporcional à altura)
3
4     AudioTrack* left_track = G(node->left->track);
5     AudioTrack* right_track = G(node->right->track);
6     int newBPM = (left_track->bpm + right_track->bpm) / 2;
7
8     printf("[Sync %d] INICIANDO SINCRONIZAÇÃO DO NOVO BPM: %d (Esq: %d, Dir: %d)\n", node->id, newBPM, left_track->bpm, right_track->bpm);
9     double parent_position = salvar_tempo_reproducao(node); // Salva o tempo de reprodução pro áudio continuar de onde parou
10    atualizar_bpm_recursivo(node, newBPM, parent_position); // Primeiro propaga o BPM para toda subárvore
11 }

```

Figura 6. Definição sincronizar_subarvore

Observe que se ignorarmos a terceira fase (atualização e replay), teremos basicamente um algoritmo que calcula a média de valores em uma árvore seguindo uma lógica bottom-up.

3.3. Exemplo Prático

Para uma entrada $k = 3$, teremos $2^{3+1} - 1 = 15$ nós. O estado inicial das threads é representado na figura abaixo.

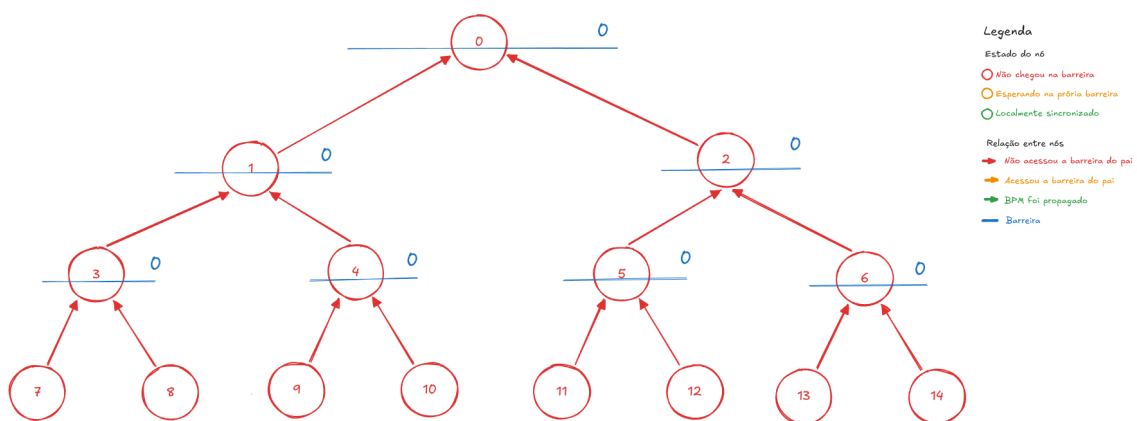


Figura 7. Exemplificação 0 sincronização.

Observe na figura abaixo que as folhas possuem uma barreira com contador 1, o código nunca acessa essas barreiras, pois as threads das folhas apenas assumem já estão localmente sincronizadas (circunferência verde), elas apenas acessam a próxima barreira da hierarquia e finalizam a própria execução. Todos nós com filhos acessam a própria barreira local (circunferência amarela), e as folhas acessaram a barreira de cada um de seus pais (seta amarela). Os nós em amarelo ainda não acessaram a barreira do próximo nó na hierarquia (seta vermelha).

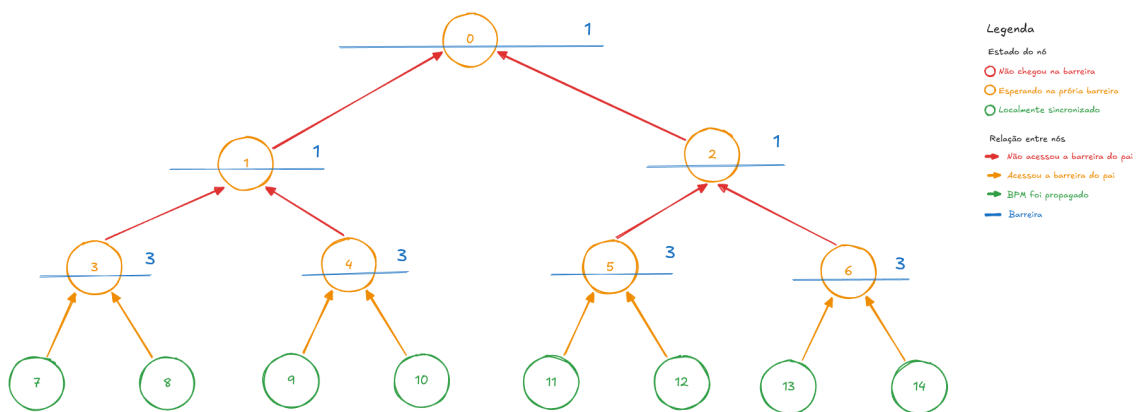


Figura 8. Exemplificação 1 sincronização.

O nó 3 fez o cálculo e propagou o novo BPM para sua subárvore e acessou a barreira do nó 1 (circunferência verde e setas verdes). O nó 6 fez o mesmo, mas não acessou a próxima barreira ainda (seta vermelha).

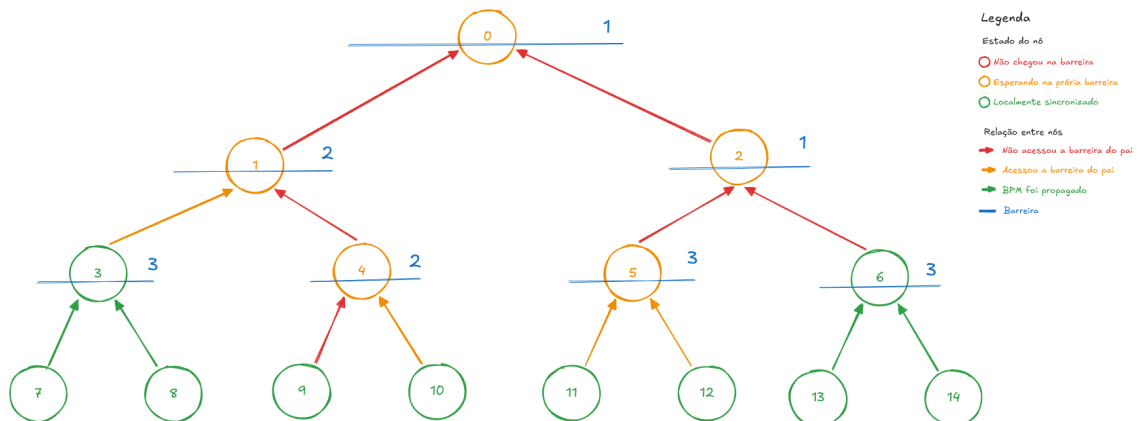


Figura 9. Exemplificação 2 sincronização.

Agora todas as subárvores cujas raízes estão no nível 2 da árvore estão sincronizadas. O nó 1 passou da própria barreira e acessou a do nó 0, o nó 2 ainda está esperando o nó 6 liberar sua barreira.

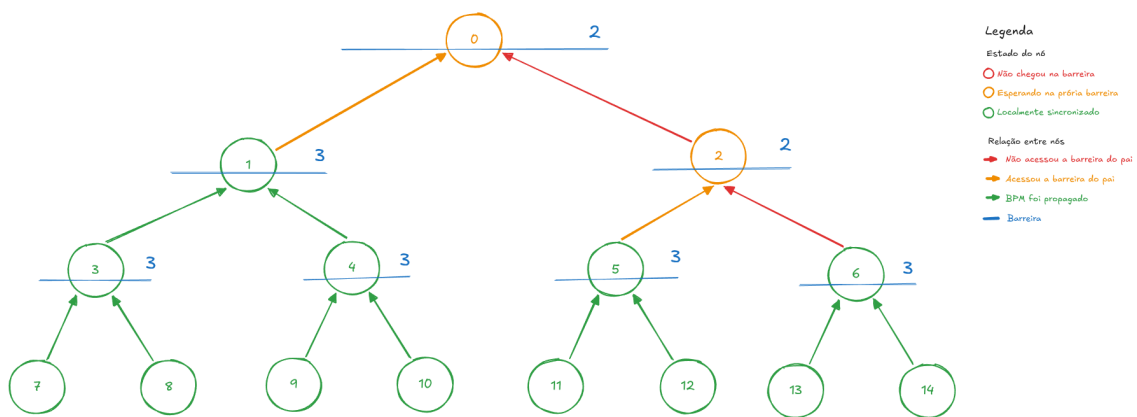


Figura 10. Exemplificação 3 sincronização.

Agora, as subárvores esquerda e direita da árvore total estão sincronizadas localmente e a barreira tem os 3 acessos. Basta a raiz da árvore calcular o BPM final e propagá-lo globalmente.

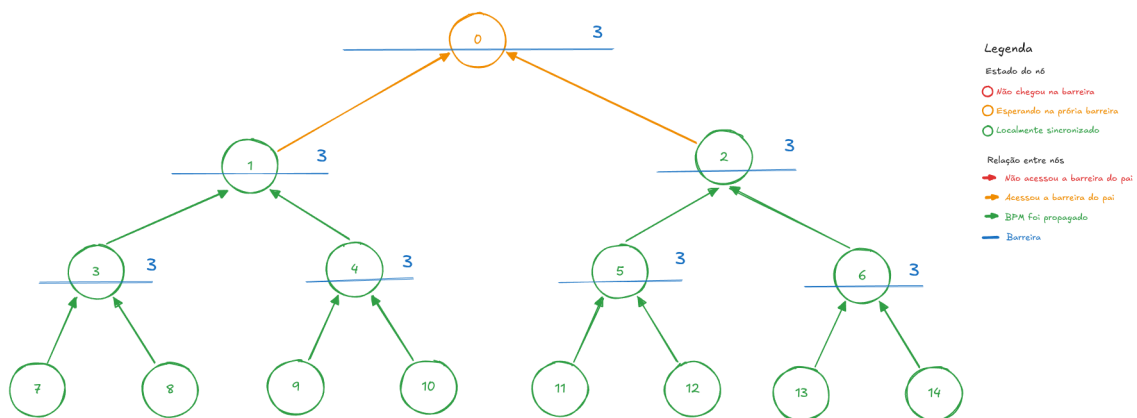


Figura 11. Exemplificação 4 sincronização.

O BPM final foi propagado para todos os nós da árvore, e todas as threads de áudio foram sincronizadas.

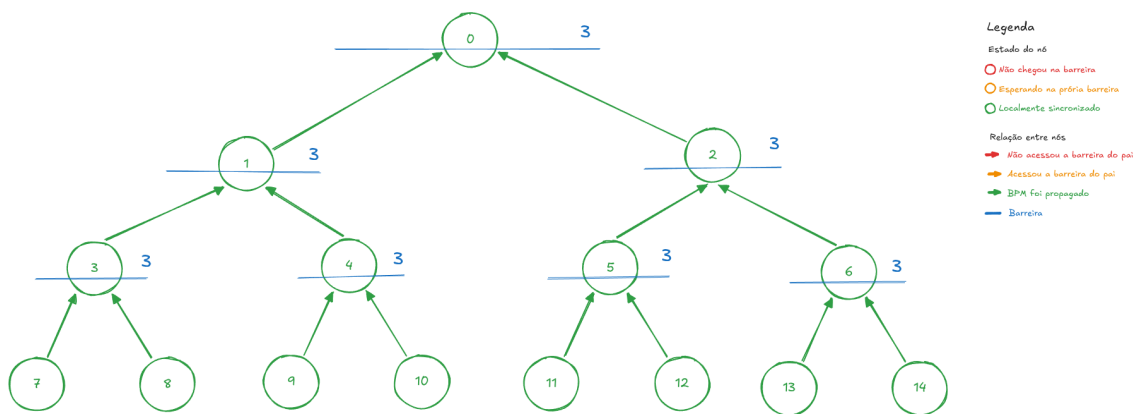


Figura 12. Exemplificação 5 sincronização.

Segue a seguir um vídeo exemplificando o processo de sincronização para árvores de diversas alturas: [Link para o vídeo](#).

4. Conclusão

O algoritmo comportou-se de acordo com o esperado na formalização do problema. No início (dependendo do número de níveis da árvore) o áudio começa caótico e com muita cacofonia. Em seguida é possível ouvir as faixas de som sincronizando-se entre si e convergindo aos poucos para um BPM médio conforme o algoritmo sobe os níveis da árvore. Quando o programa chega à raiz, é possível perceber que todas as reproduções possuem a mesma velocidade e estão tocando aproximadamente no mesmo ponto de reprodução.

Entretanto, é possível notar que existem ecos no áudio ouvido, fato que ocorre por conta da função `play_track` e do escalonamento de processos do próprio sistema operacional, que introduz um pequeno delay sempre que um comando para executar uma faixa de áudio é emitido. Isso significa que para valores de entrada maiores que 3 (que resultam em 31 ou mais threads na árvore), o eco fica cada vez mais perceptível. Esse fenômeno não fica tão óbvio para as faixas de vozes de um coral, o qual tem delays propositalmente para harmonização das vozes, mas se fizermos todas threads apontarem para uma mesma faixa de áudio de uma música qualquer esse fenômeno é mais evidente. Outra observação feita é que, devido ao crescimento exponencial do tamanho da árvore e, conseqüentemente, do número de threads, existe um problema drástico de performance.

Dito isso, pode-se afirmar que tanto o problema elaborado quanto sua solução proposta cumprem os objetivos estabelecidos pelo trabalho e ilustram um ótimo exemplo de um desafio de concorrência entre threads em uma estrutura de memória compartilhada e hierárquica em Pthreads do C.

Referências

- [1] ALCHIERI, Eduardo A. P. **Programação Concorrente - Trabalho 1: Sincronização entre Processos**. Universidade de Brasília (UnB), Instituto de Ciências Exatas, Departamento de Ciência da Computação, 2026.
- [2] THE OPEN GROUP. **POSIX.1-2017: Base Specifications**. (Especificações da biblioteca de concorrência pthread.h: pthread_barrier, pthread_mutex e pthread_cond).
- [3] LEE, Hyunkook. **3D-MARCo Project: Multimiking Comparison Recording ('A Capella')**. University of Huddersfield, 2026. Disponível em: <https://www.cambridge-mt.com/ms3/mtk/>. Acesso em: 18 jun. 2026.